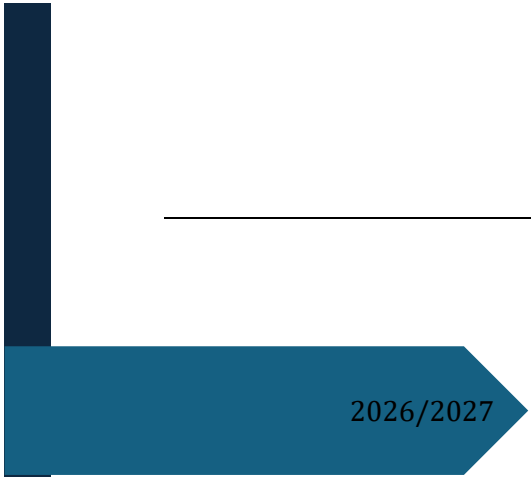
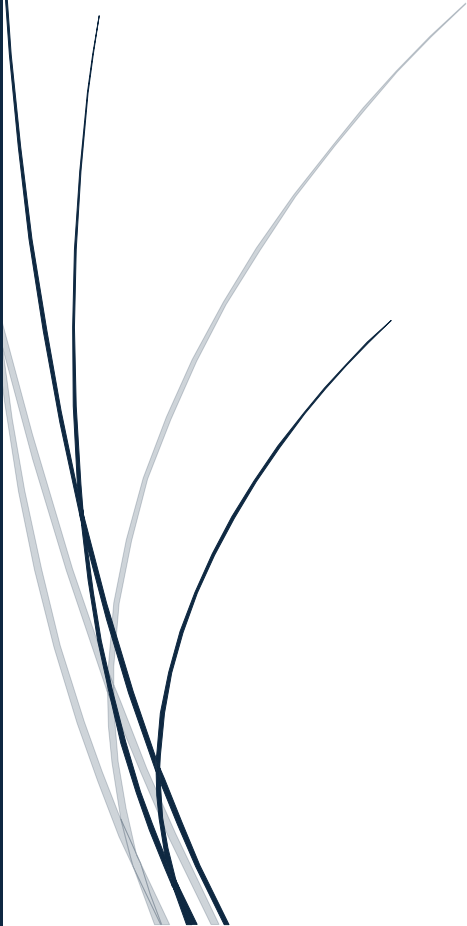


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the text '2026/2027'.

2026/2027

Desarrollo web en Entorno Servidor

DWES

Several thin, curved lines in dark blue and light grey originate from the bottom left and sweep upwards and to the right.

JUAN RIVERA ORDÓÑEZ
IES LOS SAUCES – DAW2

Desarrollo Web en Entorno Servidor.

EJERCICIOS3

1. Protocolos de comunicaciones.3
 2. Modelo de comunicaciones cliente.4
 3. Estudio sobre los métodos de petición HTTP /HTTPS más utilizados.5
 4. Estudio sobre el concepto de URI/URL/URN y relacion con HTTPS/HTTPS.6
 5. Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa.7
 6. Modelo de división funcional front-end / back-end para aplicaciones web.8
 7. Página web estática – página web dinámica – aplicación web – mashup.9
 8. Componentes de una aplicación web.10
 9. Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor - lenguajes de programación utilizados en cada caso.11
 10. Lenguajes de programación utilizados en el lado servidor de una aplicación web (características y grado de implantación actual).12
 11. Características y posibilidades de desarrollo de una plataforma XAMPP.13
 12. En qué casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entorno de explotación.14
 13. IDE más utilizados (características y grado de implantación actual).15
 14. Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual).16
 15. Apache HTTP vs Apache Tomcat.17
 16. Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual).18
 17. Generadores de documentación HTML (PHPDoc): PHPDocumentor, ApiGen.19
 18. Repositorios de software – sistemas de control de versiones: GIT , CVS, Subversión.20
 19. Propuesta de configuración del entorno de desarrollo para la asignatura de Desarrollo web del lado servidor en este curso (incluyendo las versiones): xxx-USED y xxx-WXED.21
 20. Propuesta de configuración del entorno de explotación para la asignatura de Desarrollo web del lado servidor en este curso (incluyendo las versiones): xxx-USEE.22
 21. Realizar un estudio sobre los siguientes conceptos y su relación con el desarrollo de aplicaciones web:23
- CMS – Sistema de gestión de contenidos23

ERP – Sistema de planificación de los recursos empresariales²³

22. Elegir y realizar un estudio y una presentación para la exposición del trabajo sobre una de las siguientes arquitecturas de desarrollo de Aplicaciones Web:²⁴

EJERCICIOS

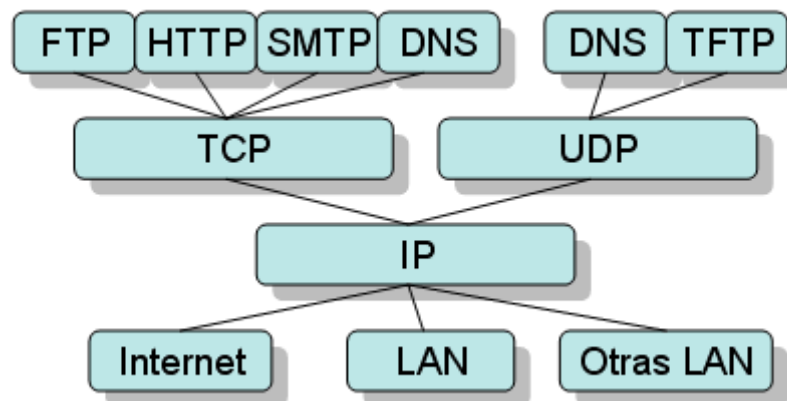
1. Protocolos de comunicaciones.

IP → Protocolo de red que proporciona direcciones lógicas y enruta los paquetes a través de la red.

TCP → Protocolo de transporte que asegura que los paquetes de datos lleguen de forma ordenada y sin errores.

HTTP → Protocolo de la capa de aplicación sin estado, utilizado para la transmisión de hipertexto. **PUERTO 80.**

HTTPS → Versión segura de HTTP que utiliza cifrado SSL/TLS para proteger los datos en tránsito. **PUERTO 443.**



Como se puede observar **HTTP/HTTPS** dependen del protocolo **TCP** para comunicarse con la red, y todos dependen del protocolo **IP** para proporcionar direcciones red para conectarse a **Internet**.

2. Modelo de comunicaciones cliente.

Arquitectura donde un programa "cliente" (como un navegador) solicita un recurso y un programa "servidor" atiende la petición y devuelve la información (archivos HTML, datos JSON, etc.). Las aplicaciones web se basan en este modelo, centralizando la lógica y los datos en el servidor.



CLIENTE -----> *Petición* -----> **SERVIDOR**

SERVIDOR -----> HTML/JSON -----> **CLIENTE**

3. Estudio sobre los métodos de petición HTTP /HTTPS más utilizados.

GET: Solicitar y obtener un recurso sin realizar ninguna modificación.

POST: Enviar datos al servidor para crear o procesar un recurso (ej. enviar un formulario).

PUT / PATCH: Actualizar recursos totales o parciales.

DELETE: Eliminar un recurso del servidor.

HEAD: Pide una respuesta como "GET" pero solo devolviendo las cabeceras.

TRACE: Solicita al servidor el mismo mensaje que ha recibido. (Bueno para ver cambios automatizados).

OPTIONS: Te devuelve la lista de *métodos de petición HTTP* que el servidor soporta.

CONNECT: Se utiliza para saber si se tiene acceso a un host.

Métodos HTTP



4. Estudio sobre el concepto de URI/URL/URN y relación con HTTPS/HTTPS.

URI: Es el término general con toda la información, contiene **URL** y **URN**.

URL: Es un tipo de **URI** que además de identificar el recurso, dice **dónde está** (dirección), **protocolo** (http/https) y como **acceder a él** (https://xxx.xxx/**donde**).

URN: Es otro tipo de **URI** que identifica al recurso por un **id UNICO**, pero no dice donde se encuentra. Ej. un ISBN → “*urn: isbn:9780141036144*”.

Su relación con HTTP es directa, ya que las URL indican al navegador a qué servidor y qué archivo exacto solicitar mediante peticiones HTTP, ya que se encuentra el protocolo escrito en el inicio de la **URI** o **URL**.

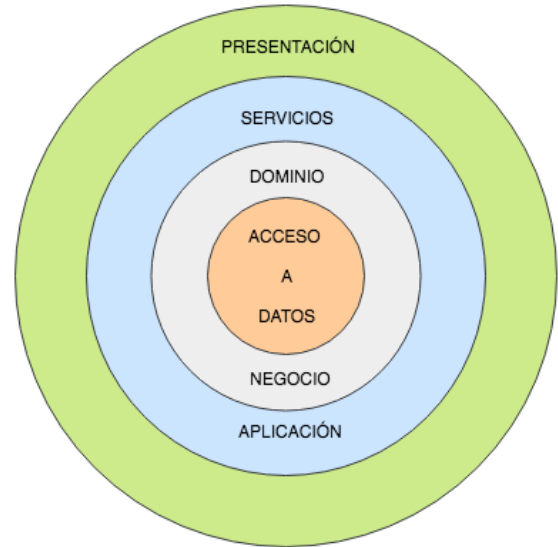


5. Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa.

Es una **arquitectura de desarrollo**, que sigue un diseño estructural para organizar el desarrollo de una aplicación bajo un principio de **separación de responsabilidades**.

El modelo estándar se compone de **tres a cuatro** capas organizadas jerárquicamente:

- **Presentación**
- **Aplicación/Servicios**
- **Dominio/Negocio**
- **Datos (BD)**



-La capa de **Presentación** captura la interacción del usuario mediante peticiones **HTTP** y es el lugar donde se devuelven los archivos **HTML** o **JSON**.

-La capa de **Aplicación** es el **controlador** en el MVC, su trabajo es coordinar la comunicación entre la capa de **presentación** y la capa de **dominio**.

-La capa de **Dominio** contiene la lógica de negocio, aquí es donde se utilizan las clases y los métodos para hacer los cálculos necesarios, también aquí es donde se ejecutan las consultas que se mandan a la **BD**.

-La capa de **Datos** es la capa encargada de alojar la base de datos, garantizando la **persistencia** del sistema y la **consistencia** de la información mediante **transacciones**.

6. Modelo de división funcional front-end / back-end para aplicaciones web.

En las aplicaciones web modernas se utiliza la división de trabajo entre dos entornos:

El **Front-End** (Lado del cliente) es el lugar donde el usuario final interactúa con la aplicación y manda peticiones **HTTP**.

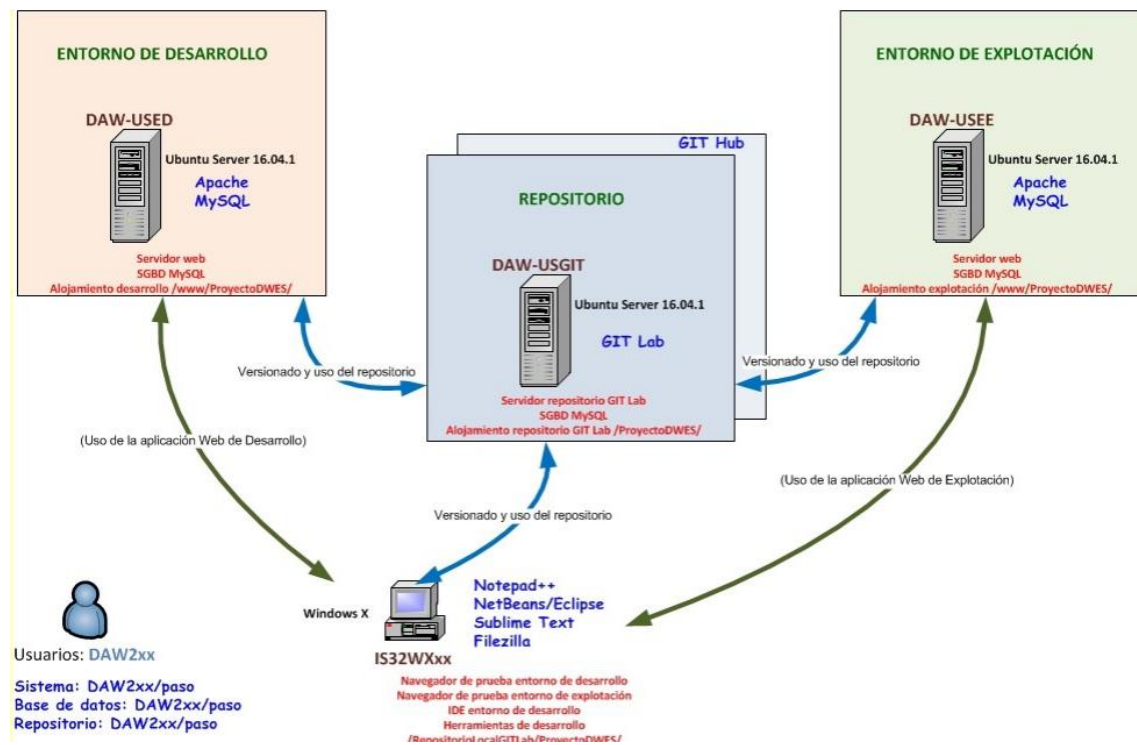
El **Back-End** (Lado del servidor) es el lugar donde residen los recursos, la lógica de programación y la base de datos.

¡¡CABE DESTACAR!!

Esta separación solo interpreta el uso de los recursos en cada lado, pero los ficheros y datos de ambas capas se guardan en el **servidor web**, el cual se encuentra en el **Back-End** (Entorno Servidor).

<u>FRONT-END</u>	<u>RED</u>	<u>BACK-END</u>
• HTML5 • CSS3 • JavaScript	→Peticiones HTTP/HTTPS Archivos HTML/JSON ←	• Php, Java, etc... • SGBD • BD

8. Componentes de una aplicación web.



Una **aplicación web** tiene todos estos componentes:

- Un servidor web
- Un SGBD
- Una BD
- Un intérprete de código
- Ficheros/código escrito por nosotros

9. Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor - lenguajes de programación utilizados en cada caso.

Los programas ejecutados en el lado del **cliente** se ejecutan localmente en el dispositivo del usuario dentro del **navegador web**.

Se utiliza principalmente:

- **HTML5**
- **CSS3**
- **JavaScript/TypeScript**

Los programas ejecutados en el lado del **servidor** se ejecutan en el servidor y estos son alojados por un **servidor web**.

Se utiliza principalmente:

- **MySQL/MongoDB**
- **PHP, Java, Python, JavaScript (Node.js), etc.**

10. Lenguajes de programación utilizados en el lado servidor de una aplicación web (características y grado de implantación actual).

CLASIFICACIÓN POR GRADO DE IMPLANTACIÓN

1. **Java**

Es el estándar para el software empresarial, financiero e institucional actualmente.

Es un lenguaje con **tipado fuerte**, además es **compilado**.

2. **JavaScript/TypeScript**

Node.js permitió unificar el lenguaje del lado de **cliente** y **servidor** bajo un mismo lenguaje, es el estándar para **APIs REST** ligeras y arquitecturas **BFF**.

3. **Python**

Es el estándar para el entrenamiento de **IAs** y el **Machine Learning**.

Tiene **tipado dinámico** además de ser **interpretado**.

4. **PHP**

Es un lenguaje muy alto en cuota global ya que es el que sustenta un porcentaje enorme de la **web visible** (en gran parte debido a **WordPress**).

Es un lenguaje también **interpretado**.

5. **.NET**

Es un gran estándar en el **ámbito empresarial** rivalizando directamente con **Java**.

Se dice que es uno de los lenguajes con **mayor rendimiento** y cómodo de trabajar al contar con un **ecosistema integrado** en el **ASP.NET Core**.

11. Características y posibilidades de desarrollo de una plataforma XAMPP.

XAMPP es un paquete de distribución de software libre multiplataforma creado para proporcionar un **entorno de servidor local listo para usar en un solo instalador**.

X → Multiplataforma

A → Apache HTTP Server

M → MySQL

P → PHP

P → Perl

Lo bueno de **XAMPP** es que es un entorno todo en uno preconfigurado, muy bueno para **probar, desarrollar o estudiar**, permite desarrollo **back-end** con **PHP**, una base de datos relacional gestionada en **phpMyAdmin**, buena integración con **CMS**.

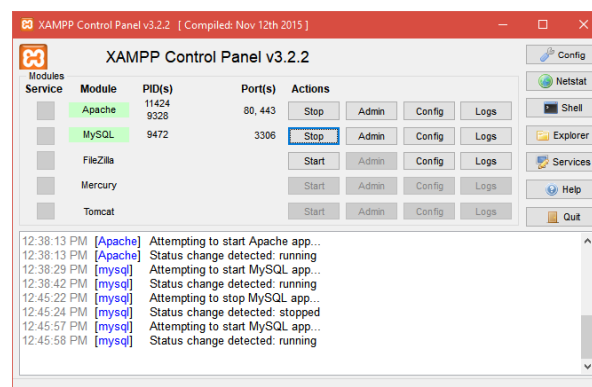
Lo malo de **XAMPP** es que tiene una configuración de seguridad prácticamente **nula**, no está hecho para recibir optimizaciones de rendimiento y le falta **aislamiento de procesos**.

Entonces en la pregunta:

¿Es XAMPP un buen entorno de alojamiento de servidor en producción?

NO

La respuesta corta y categórica es:



12. En qué casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entorno de explotación.

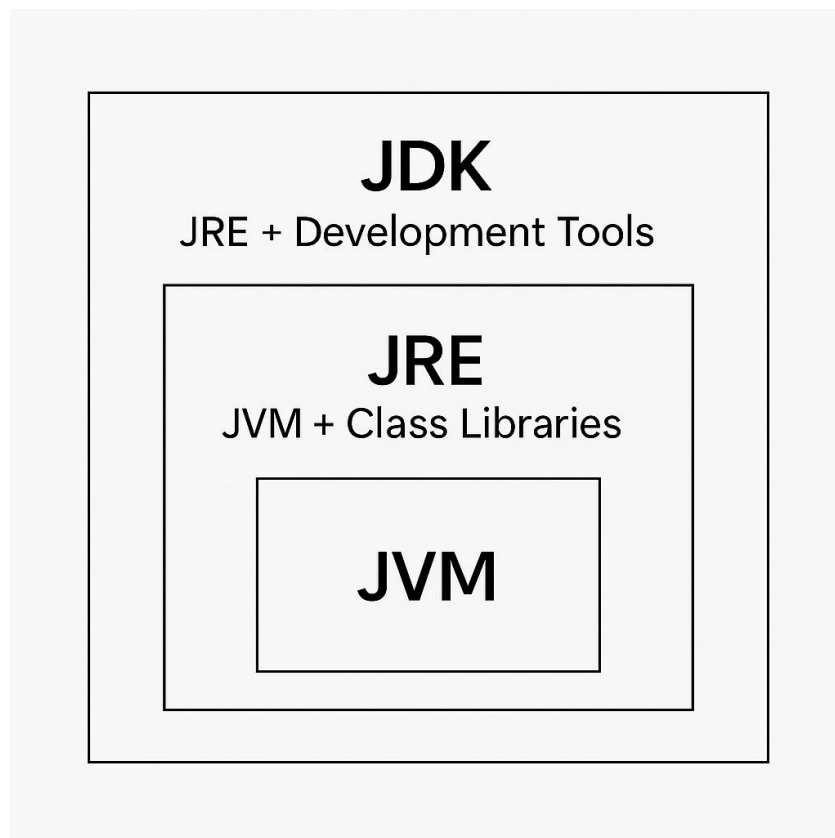
Por regla general:

- En el **entorno de desarrollo** es necesario tanto el **JVM** como el **JDK**.

Se requiere el compilador (**javac**) para crear los **".class"**, las librerías de prueba y el **JVM** para verificar el funcionamiento.

- En **producción** es estrictamente obligatoria la **JVM** pero no es necesario el **JDK**.

En producción solo se ejecutan **archivos precompilados ".jar"**, no se compila en el servidor. Excluir el **JDK** mínima la superficie de ataque y el consumo de disco.



13. IDE más utilizados (características y grado de implantación actual).

CLASIFICACIÓN POR GRADO DE IMPLANTACIÓN

1. Visual Studio Code (VS Code)

Es con diferencia el **líder absoluto y universal**, es el estándar indiscutible para desarrollo en **HTML, CSS, JavaScript, Python, C++ y C#**.

2. IntelliJ IDEA

Es el entorno de referencia para el desarrollo en el ecosistema **Java y JVM**.

Permite un grado de **indexación profundo** y tiene integración nativa con Gradle y Maven.

3. Visual Studio (Microsoft)

Es el IDE tradicional de **Microsoft**. Está optimizado para el ecosistema **.NET** y desarrollo nativo de alto rendimiento en **Windows**.

4. Eclipse

Fue el rey indiscutible en desarrollo **Java** y pionero de los entornos con integración de **plugins**. Actualmente se usa en sectores industriales y universidades debido a su uso **gratuito**.



14. Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual).

CLASIFICACIÓN POR GRADO DE IMPLANTACIÓN

1. Nginx

Basado en **eventos** y programación **asíncrona**. Es un excelente servidor para el manejo de entregas de contenido **estático**.

2. Apache HTTPd

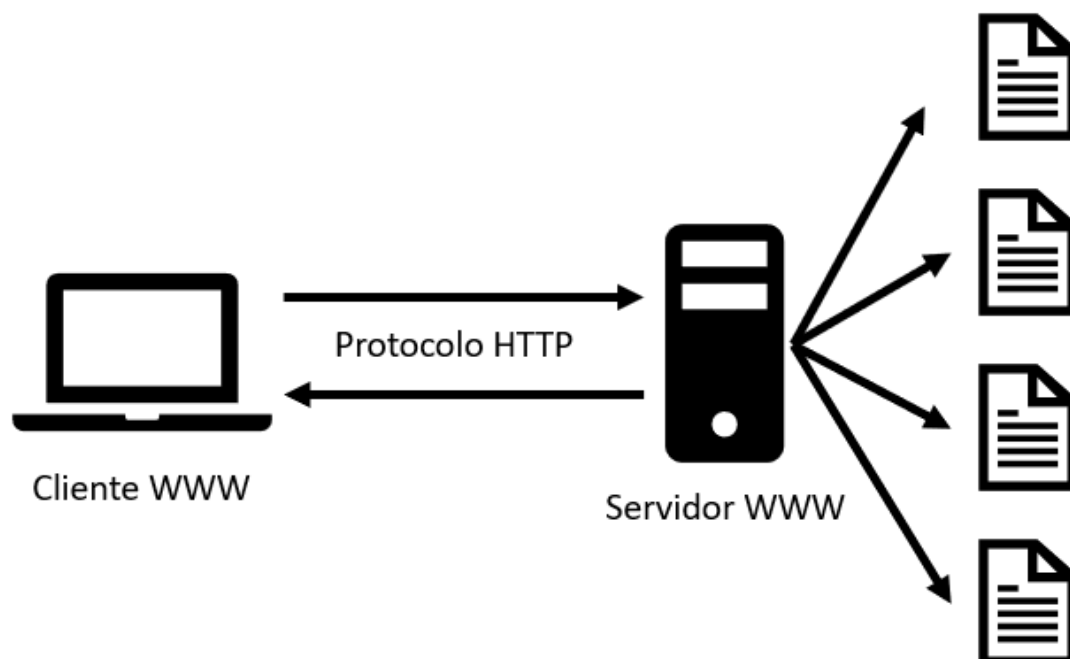
Orientado al uso de **procesos/hilos** permite que cada conexión sea asignada a un proceso individual. Es actualmente el servicio más **flexible** que hay.

3. Cloudflare Server

Es una infraestructura basada en servidores **perimetrales**, siendo este un entorno más seguro contra ataques de tipo **DDoS**.

4. Microsoft IIS

Es el **servidor web** nativo de **Microsoft**, integrado directamente en **Windows Server**. Como todos los servicios de esta compañía, lidera en grandes empresas, entidades bancarias e instituciones públicas.



15. Apache HTTP vs Apache Tomcat.

Aunque compartan el nombre “Apache” en el nombre son fundamentalmente distintos:

Apache HTTP Server es un **servidor web**, mientras que Apache Tomcat es un **contenedor de servlets de Java**.

	<u>Apache HTTP Server</u>	<u>Apache Tomcat</u>
Categoría	Servidor Web	Contenedor de Servlets
Lenguaje en que está programado	C	Java (se ejecuta sobre la JVM)
Dependencia de la JVM	No	Sí
Especialidad principal	Contenido estático (HTML, CSS, JS, multimedia).	Contenido dinámico generado por clases Java (Servlets, JSPs, APIs Spring)
Empaquetado que despliega	Ficheros planos en directorios (htdocs, /var/www/html)	Paquetes empaquetados .war en el directorio webapps
Puerto por defecto	80 (HTTP) / 443 (HTTPS)	8080
Módulos / Extensiones	Compilados en C (mod_rewrite, mod_ssl, mod_proxy)	Librerías Java (.jar), servlets y filtros definidos en web.xml

16. Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual).

CLASIFICACIÓN POR GRADO DE IMPLANTACIÓN

1. **Chrome**

Es con diferencia el navegador más utilizado en el mundo. Utiliza el motor de renderizado **Blink**. Funciona con una arquitectura **multiproceso estricta**: cada pestaña corre en un proceso independiente.

2. **Safari**

Es el navegador por defecto del ecosistema **Apple**. Utiliza **Webkit**. Diseñado para aprovechar la aceleración por hardware de los chips de **Apple**.

3. **Edge**

Es el sustituto oficial de **Internet Explorer**, utiliza también **Blink**. Tiene integración nativa con **Windows**, aprovecha mejor la RAM utilizando *"Sleeping Tabs"*

4. **Firefox**

Es el único navegador de escritorio (de los grandes) que es completamente independiente de **Chromium**.

Esta renderizado por **Gecko**, mantenido por **Mozilla**.

Es un navegador de código abierto, muy querido por la comunidad de administradores de sistemas **Linux**.

17. Generadores de documentación HTML (PHPDoc): PHPDocumentor, ApiGen.

18. Repositorios de software – sistemas de control de versiones: GIT , CVS, Subversión.

19. Propuesta de configuración del entorno de desarrollo para la asignatura de Desarrollo web del lado servidor en este curso (incluyendo las versiones): xxx-USED y xxx-WXED.

20. Propuesta de configuración del entorno de explotación para la asignatura de Desarrollo web del lado servidor en este curso (incluyendo las versiones): xxx-USEE.

21. Realizar un estudio sobre los siguientes conceptos y su relación con el desarrollo de aplicaciones web:

CMS – Sistema de gestión de contenidos

ERP – Sistema de planificación de los recursos empresariales

22. Elegir y realizar un estudio y una presentación para la exposición del trabajo sobre una de las siguientes arquitecturas de desarrollo de Aplicaciones Web:

- MEAN (con MongoDB y con MySQL)
- Java EE vs Spring
- Microsoft .NET
- Angular 7
- Symfony
- Laravel
- CakePHP
- CodeIgniter

GLOSARIO DE TÉRMINOS RELACIONADOS CON DWES

- Contenidos y la diferencia entre los módulos que tienes en este curso.
- Protocolos TCP/IP. Socket.
- Protocolo HTTP / HTTPS
- HTML
- XML
- JSON
- Lenguajes de programación embebidos en HTML ➤ Arquitecturas de desarrollo web
- Framework de desarrollo Web
- ERP
- CMS
- PHP
- IDE
- Navegador
- Repositorio
- Entorno de Desarrollo
- Entorno de Explotación o Producción
- Gestión de la configuración. Control de cambios. Mantenimiento de la aplicación.
- Web Services
- AJAX
- Desarrollo de aplicaciones multicapa. Estrategias de diseño de aplicaciones Web.
- Aplicaciones basadas en microservicios

- SaaS: Software as a Service
- Control de acceso a la aplicación web o los Web Services
- Validación de entrada de datos a una aplicación Web
- Posicionamiento de una aplicación Web
- Historia, situación actual y evolución del diseño de aplicaciones Web